



LFM2.5-2.6B Model Exploration Writeup

Context:

- **Hardware:** MacBook Air, Apple M2 chip, 16 GB unified memory
- **Runtime:** llama.cpp
- **Model:** LFM2.5-2.6B Q4_K_M GGUF

Part 1: Hands on

I ran the quantized LFM2.5-2.6B model locally on my MacBook Air using `llama.cpp` and the official `Q4_K_M` GGUF checkpoint. This satisfied the local-inference requirement without using a cloud GPU. I first tested the model interactively through `llama cli` with short prompts such as:

Explain what a purchase order is in one sentence.

What is 15% of 280?

For these short prompts, generation throughput was approximately 43 to 46 tokens per second. One run showed `Prompt: 42.2 t/s | Generation: 46.2 t/s`, while another showed `Prompt: 93.9 t/s | Generation: 43.4 t/s`. During this workload, the `llama` process used approximately 2.11 GB of memory according to macOS Activity Monitor.

```
liquid_part1 — zsh — 99x57

Loading model...

llama.cpp

build      : b10809-5266f24da
model      : LiquidAI/LFM2.5-2.6B-GGUF:Q4_K_M
ftype      : Q4_K - Medium
modalities : text

available commands:
  /exit or Ctrl+C    stop or exit
  /regen             regenerate the last response
  /clear             clear the chat history
  /read <file>       add a text file
  /glob <pattern>    add text files using globbing pattern

>

[> Explain what a purchase order is in one sentence. ]

[Start thinking]

The user wants a one-sentence explanation of what a purchase order is. I need to be concise and accurate. A purchase order is a document used by a buyer to request goods or services from a seller, outlining the terms and quantities. I'll craft a single sentence that captures this.

[End thinking]

A purchase order is a formal document issued by a buyer to a seller that specifies the items, quantities, prices, and delivery terms, serving as a binding request for goods or services.

[ Prompt: 42.2 t/s | Generation: 46.2 t/s ]

[> What is 15% of 280? ]

[Start thinking]

The user is asking for a simple math calculation: 15% of 280. I need to calculate this. 15% of 280 = 0.15 * 280. Let me compute: 0.15 * 280 = 42. So the answer is 42. I should provide a clear, direct answer.

[End thinking]

15% of 280 is 42.

[ Prompt: 93.9 t/s | Generation: 43.4 t/s ]
```

Long-context test with a realistic text file (sensitive information blurred)

I then tested a much larger input using a billing dispute context that I had previously worked through. After loading the file, I asked: “Based only on the context I just loaded, summarize the billing dispute and calculate how much new cash I actually need to pay after accounting for the £XXXX retainer. Explain any ambiguity you find.” This longer-context run produced `Prompt: 448.9 t/s | Generation: 25.3 t/s`, while process memory increased only slightly, from approximately 2.11 GB to 2.13 GB. The most noticeable effect of the longer context was therefore not memory growth, but a substantial reduction in generation speed.

```
Loading model...
```



```
build      : b10809-5266f24da
model      : LiquidAI/LFM2.5-2.6B-GGUF:Q4_K_M
ftype     : Q4_K - Medium
modalities : text
```

```
available commands:
/exit or Ctrl+C    stop or exit
/regen             regenerate the last response
/clear            clear the chat history
/read <file>       add a text file
/glob <pattern>    add text files using globbing pattern
```

```
> /read /Users/zimoshan/Downloads/legal_fee_dispute_context.txt
Loaded text from '/Users/zimoshan/Downloads/legal_fee_dispute_context.txt'
```

```
> Based only on the context I just loaded, summarize the billing dispute and calculate how much new cash I actually need to pay after accounting for the £1,000 retainer. Explain any ambiguity you find.
```

```
[ Prompt: 448.9 t/s | Generation: 25.3 t/s ]
```

Test	Prompt	Memory	Prompt throughput	Generation throughput
Short	“Explain what a purchase order	~2.11 GB	42.2 t/s	46.2 t/s

Test	Prompt	Memory	Prompt throughput	Generation throughput
	is in one sentence.”			
Short	“What is 15% of 280?”	~2.11 GB	93.9 t/s	43.4 t/s
Long context	billing dispute file + analysis question	~2.13 GB	448.9 t/s	25.3 t/s

llama-server	2.11 GB	16	76	19297	zimoshan
--------------	---------	----	----	-------	----------

Activity Monitor - snapshot 1

llama-server	2.18 GB	17	74	19297	zimoshan
--------------	---------	----	----	-------	----------

Activity Monitor - snapshot 2

Experiment	Result
Short-context memory	~2.11 GB
Long-context memory	~2.13 GB
Short-context generation	~43 to 46 tok/s
Long-context generation	25.3 tok/s

Tool Calling:

For tool use, I ran LFM2.5-2.6B behind a local `llama-server` and exposed a Python function, `lookup_purchase_order(po_number)`, backed by a small mock purchase-order database. I prompted the model with an invoice for \$12,450 that referenced `P0-4821`. The model selected `lookup_purchase_order` and independently populated the argument `{"po_number": "P0-4821"}` from the natural-language context. My Python runtime then executed the actual function, returned the purchase-order record showing an amount of \$12,000, and passed the result back to the model. The model then correctly identified the \$450 invoice-to-PO discrepancy.

I used the OpenAI Python client only as an API wrapper for llama.cpp's OpenAI-compatible local endpoint at `127.0.0.1:8080`; no inference request was sent to OpenAI or another cloud provider.

```

zima: ~ % llama-server \
(base) zimoshan@Zimos-MacBook-Air ~ % llama-server \
  -hf LiquidAI/LFM2.5-2.6B-GGUF:Q4_K_M \
  --jinja \
  --alias lfm2.5-2.6b \
  --port 8080
0.00.832.009 I cmn common_param: common_params_print_info: verbosity = 3 (adjust with the '-lv N' CLI arg)
0.00.833.986 W srv llama_server: -----
0.00.833.992 W srv llama_server: CORS is set to allow all origins ('*') and no API key is set
0.00.833.993 W srv llama_server: this can be a security risk (cross-origin attacks)
0.00.833.994 W srv llama_server: more info: https://github.com/ggml-org/llama.cpp/pull/25655
0.00.833.994 W srv llama_server: -----
0.00.836.380 I srv load_model: loading model 'LiquidAI/LFM2.5-2.6B-GGUF:Q4_K_M'
0.02.037.687 I cmn init: llama threadpool init, n_threads = 4
0.02.232.066 I srv load_model: initializing, n_slots = 4, n_ctx_slot = 131072, kv_unified = 'true'
0.02.237.020 W srv init: chat template supports preserving reasoning, it is enabled by default (may use more
tokens, disable via --no-reasoning-preserve)
0.02.237.030 I srv llama_server: model loaded
0.02.237.038 I srv llama_server: listening on http://127.0.0.1:8080
0.02.237.039 W srv llama_server: NOTICE: server default port will be changed to :9931 in a future release
0.02.237.039 W srv llama_server: ref: https://github.com/ggml-org/llama.cpp/pull/26508
[13.33.923.379 I slot get_avaiabl: id 3 | task -1 | selected slot by LRU, t_last = -1
13.33.925.020 I slot launch_slot_: id 3 | task 0 | processing task, is_child = 0
13.37.924.557 I slot print_timing: id 3 | task 0 | prompt eval time = 1921.85 ms / 129 tokens ( 14.90 ms per t
oken, 67.12 tokens per second)
13.37.924.560 I slot print_timing: id 3 | task 0 | eval time = 2076.80 ms / 87 tokens ( 24.15 ms per t
oken, 41.41 tokens per second)
13.37.924.560 I slot print_timing: id 3 | task 0 | total time = 3998.65 ms / 216 tokens
13.37.924.561 I slot print_timing: id 3 | task 0 | graphs reused = 86
13.37.931.122 I slot release: id 3 | task 0 | stop processing: n_tokens = 215, truncated = 0
13.38.048.983 I slot get_avaiabl: id 3 | task -1 | selected slot by LCP similarity, f_sim_best = 0.867 (> 0.100 thol
d), f_keep = 1.000

```

Terminal of my personal Mac

```

def lookup_purchase_order(po_number):
    return purchase_orders.get(po_number)

```

```

tools = [{
    "type": "function",
    "function": {
        "name": "lookup_purchase_order",
        ...
    }
}]

```

For complete Python code, please see below file:

tool_test.py

Output:

Tool selected:

lookup_purchase_order

Arguments generated by model:

{"po_number": "P0-4821"}

Actual Python function result:

{"vendor": "ABC Logistics", "amount": 12000, "currency": "USD"}

```
(.venv) (base) zimoshan@Zimos-MacBook-Air liquid_part1 % python3 tool_test.py
First model call (tool selection + argument generation): 2.63 seconds
Raw model response:
ChatCompletionMessage(content='', refusal=None, role='assistant', annotations=None, audio=None, function_call=None, tool_calls=[ChatCompletionMessageFunctionToolCall(id='mGH0XfyhEaTfQTLqnTg82h8dCKFVct60', function=Function(arguments={'po_number': 'P0-4821'}, name='lookup_purchase_order'), type='function')], reasoning_content='The user wants me to check purchase order P0-4821 to identify any discrepancy with an invoice for $12,450. I need to look up the purchase order first to see its details and compare with the invoice amount. Let me use the lookup_purchase_order tool with the PO number provided.')
Tool selected:
lookup_purchase_order
Arguments generated by model:
{'po_number': 'P0-4821'}
Actual Python function result:
{'vendor': 'ABC Logistics', 'amount': 12000, 'currency': 'USD'}
Second model call (tool-result interpretation + final generation): 2.22 seconds
Final answer:
The purchase order P0-4821 from ABC Logistics has an amount of $12,000, while the invoice is for $12,450. There is a discrepancy of $450 between the purchase order amount and the invoice amount.
Finish reason:
stop
Usage:
CompletionUsage(completion_tokens=93, prompt_tokens=301, total_tokens=394, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=268, image_tokens=None, text_tokens=None))
Total end-to-end workflow latency: 4.86 seconds
Approximate non-model orchestration overhead: 0.00 seconds
(.venv) (base) zimoshan@Zimos-MacBook-Air liquid_part1 %
```

Terminal View of Curosr

- First model call: 2.63 s
- Second model call: 2.22 s
- End-to-end workflow: 4.86 s
- Finish reason: stop

Workflow:



User request



LFM selects tool + generates arguments (2.63 s)



Python function executes

↓

LFM interprets tool result + generates final response (2.22 s)

↓

Final answer (Total end-to-end latency = 4.86 s)

Test	Memory	Prompt throughput	Generation throughput	E2E latency
Short, purchase order	~2.11 GB	42.2 tok/s	46.2 tok/s	
Short, math	~2.11 GB	93.9 tok/s	43.4 tok/s	
Long billing context	~2.13 GB	448.9 tok/s	25.3 tok/s	
Tool workflow	~2.11 GB			4.86 s

In this run, essentially all end-to-end latency came from the two local model inference calls; Python lookup, JSON parsing, and localhost orchestration contributed negligible overhead.

What surprised me:

- What surprised me most was how practical a 2.6B-parameter model was on a consumer laptop after quantization.
 - I expected that inference would feel much more resource constrained on local hardware.
 - Instead, the Q4 model used only about 2.1 GB of process memory, and could generate short responses at around 44 tokens per second.
- I was also surprised by how little configuration was required to get to a working local model. I had expected to encounter much more low-level configuration in getting to a working local model, but with the GGUF checkpoint and llama.cpp, I was able to get LFM2.5-2.6B running on my MacBook Air relatively quickly. I still don't fully understand the model's operating boundaries or how best to leverage it, but I was able to get to a usable local inference loop much faster than I expected.

- I was also surprised that the longer billing-dispute context only modestly changed the reported process memory, but dropped generation speed from roughly 43–46 tokens per second to about 25 tokens per second. My initial intuition was that longer context lengths would first manifest themselves as a large increase in memory usage. In my experiment, the impact showed up much more clearly in generation throughput than in the coarse process-memory measurement.
- The tool-calling loop was also simpler conceptually than I expected once I built it end to end. The model didn't need direct access to the backend. It just had to select the correct function and generate structured arguments from context, and let Python perform the actual execution. Seeing the model extract PO-4821 from natural language, call the lookup function, and then reason over the returned record made the separation between model intelligence and deterministic application logic much more concrete.

What was harder than expected:

- Understanding what the runtime metrics actually meant was harder than I expected. `llama.cpp` reports prompt-processing throughput and generation throughput separately, and those numbers behave differently.
 - For example, the longer input had a much higher prompt-processing rate in tokens per second than the short prompts, even though the overall interaction felt slower. This made it clear that tokens per second alone is not the same thing as end-to-end latency. Short prompts are more affected by fixed overhead, while a long prefill can make better use of the hardware.
 - Generation speed, on the other hand, was clearly slower with the larger context. I also found that process memory from Activity Monitor is only an approximate system-level measurement rather than a clean decomposition of model weights, KV cache, and runtime buffers.
- Tool calling was also more explicit than I initially expected. The model does not execute Python directly. It selects a function and generates structured arguments; the Python runtime executes the function, and the result is then passed back to the model for another inference step. Building this loop made the boundary between probabilistic model behavior and deterministic application logic much clearer. It

also showed me that an apparently simple tool-using interaction can require multiple model inference calls, which directly contributes to end-to-end latency.

What I still do not fully understand:

- How the observed process memory decomposes across quantized model weights, KV cache, preallocated runtime buffers, and other `llama.cpp` overhead.
- How memory usage and decode throughput scale as context length grows much larger.
- How much Q4 quantization affects model quality, particularly tool selection and argument-generation reliability, relative to higher-precision checkpoints.
- I still do not fully understand why some generations stopped partway through the final answer. Increasing `max_tokens` substantially, and even removing the explicit limit, did not consistently prevent the truncation. I eventually obtained a clean completion with `finish_reason="stop"` after tightening the prompt and changing the serving setup, but I do not yet understand how much of the earlier behavior came from hidden reasoning-token usage, the chat template, server-side generation settings, stop-token handling, or another interaction between the runtime and the model.

Part 2

llama-server	3.23 GB	16	93	19297	zimoshan
--------------	---------	----	----	-------	----------

Activity Monitor snapshot after Part 2 agent building

results in a nutshell

see github repo